

SAS Intelligent Decisioning

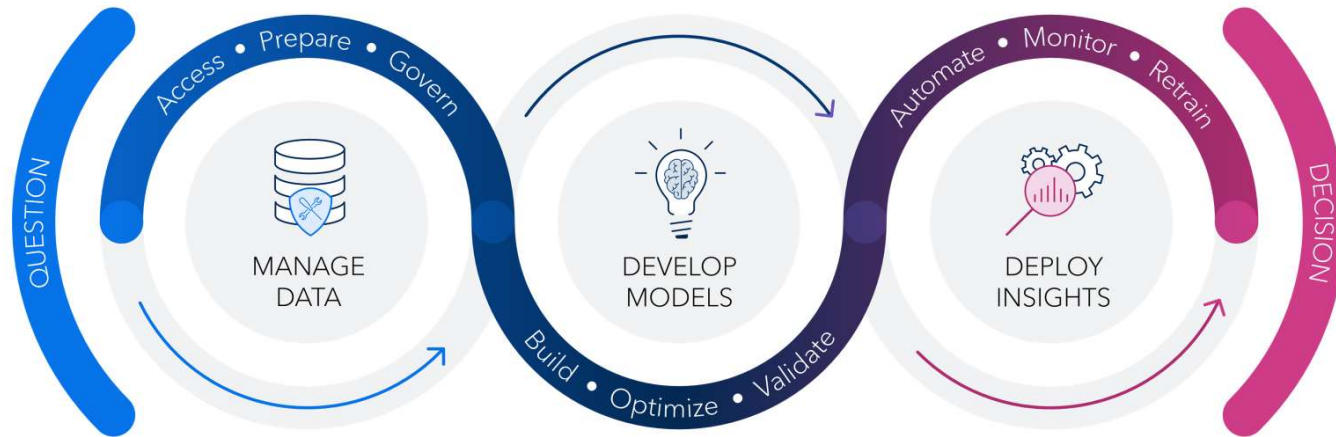
Beyond Out of the Box

Clemens Knobloch - SAS Global Technology Practice



Decisioning Across the Entire AI Lifecycle

End to End Lifecycle Support



Data for Analytics
and Decisioning

Analytical
Visualization

Advancing with
AI

Automate
Modeling

Operationalizing
Analytics

*Accelerate data storytelling
and increase speed to value.*

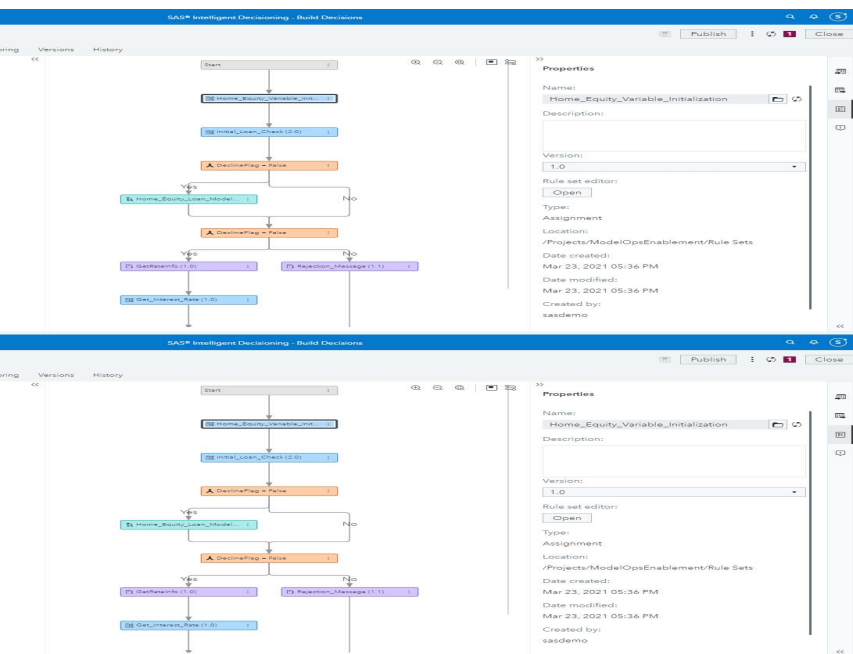
*Thread automation and
intelligence throughout the
analytics life cycle.*

*Deliver fast, self-service power to
every user—and deliver embedded
smart insights.*

DataOps + ModelOps + DecisionOps

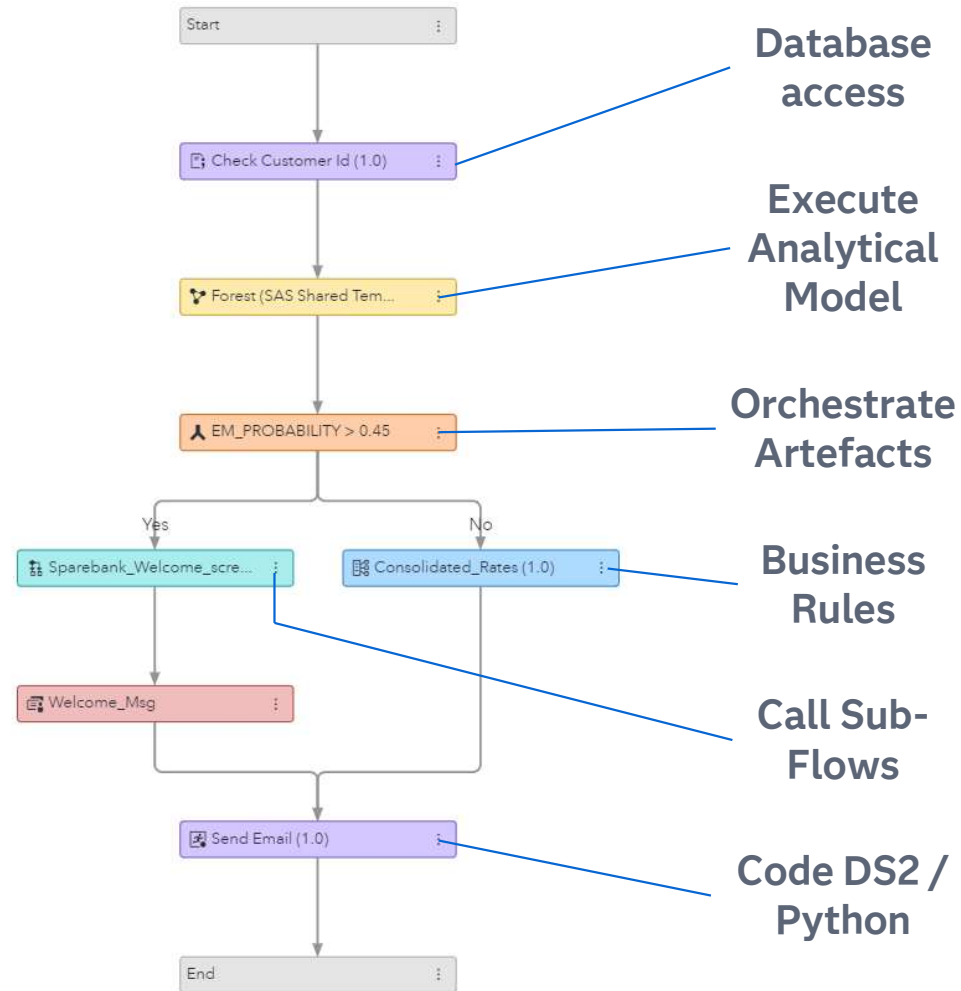
SAS® Intelligent Decisioning

Operationalize Analytics



- Build decisions and rules via drag and drop interface
- Test Decision inside Viya environment
- Govern decision development through workflow and versioning
- High performant real-time execution via REST API

SAS® Intelligent Decisioning



SAS® Intelligent Decisioning

The screenshot displays the SAS Intelligent Decisioning - Build Decisions interface. The main window shows the 'Discount (1.0)' decision with tabs for Decision Flow, Decision, Properties, Variables, Scoring, Versions, and History. The 'Variables' tab is active, showing a table of decision variables:

Decision Variables	Global Variables	
Name	Data Type	Input
Policies	Data grid	☒
PolicyHolder	Character	☒

An 'Edit Variable' dialog box is open, showing a dropdown menu for 'Data type' with options: Binary, Boolean, Character, Data grid, Date, Datetime, Decimal, Integer, Varying-length binary, and Data grid (selected). The dialog also includes fields for Name, Description, and Data type, and a checkbox for 'Data does not contain column metadata'. The background workflow diagram shows a sequence of steps: 'Welcome_Msg' (red box) -> 'Send Email (1.0)' (purple box) -> 'End' (grey box). A blue arrow points from the 'Send Email (1.0)' step to the text 'Code DS2 / Python'.

Copyright © SAS Institute Inc. All rights reserved.

SAS® Intelligent Decisioning

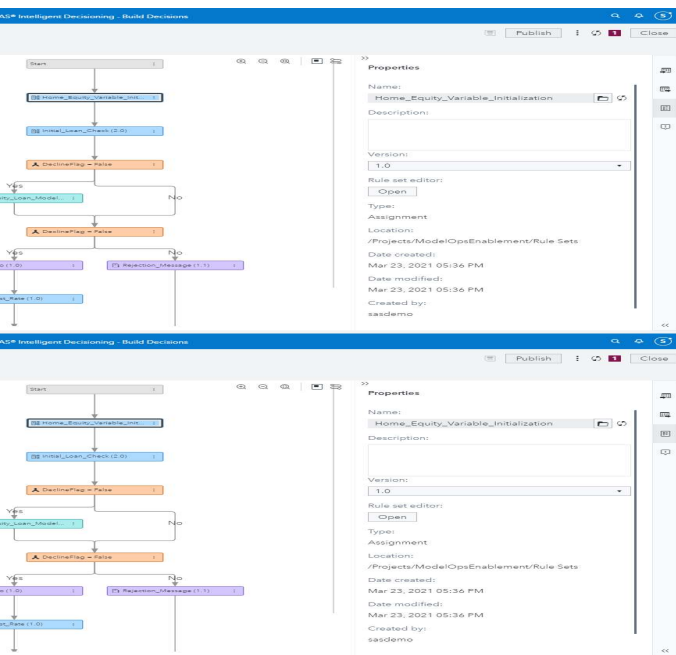
Data Grids

PolicyHolder	Policies	
Smyth, Joe	PolicyNumber	YearlyPremium
	453975R398	439.50
	987348P210	132.90
Dupree, Marcel	PolicyNumber	YearlyPremium
	983092B228	334.00
	274933P412	219.35

What is a data Grid?

- A table on a decision flow
- Decision flow variable of type DATAGRID
- Stored in JSON format in DATAGRID variable
- 100+ OOO Data Grid functions
 - Create, Update, or Delete
 - Retrieve Values
 - Set Values
 - Statistical

SAS® Intelligent Decisioning Data Grids

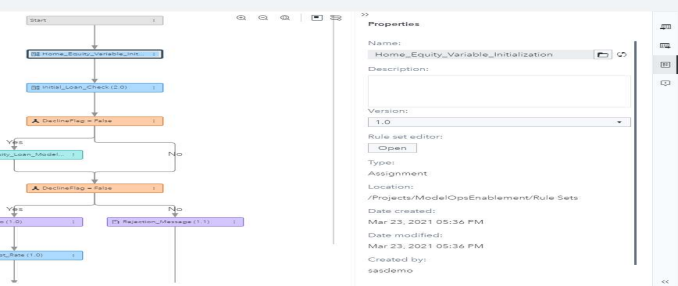


- 100+ 000 Data Grid functions
 - Laborious for intense operation
 - Can be too slow on big Data Grids
- Using DuckDB
 - Custom function calling Python code for DuckDB
 - Run more sophisticated operation with one statement
 - E.g.: Creating pivot table
 - E.g.: Using SQL aggregate functions
 - More intense queries are quicker
 - Data Grid function ~ 0.000002 sec
running 5M times ~ 10 sec

Search		🔍	Filter ▾
	Name	Description	Category
☐	redis_connect	Retrieve Redis ...	Redis
☐	tw_getdata	Search and ...	Redis
☐	tw_setdata	Store data in ...	Redis
☐	datagrid_sql_pg	Use SQL to que...	dataGrid
☐	datagrid_sql	Use SQL to que...	dataGrid

Modified By

Decisioning



Copyright © SAS Institute Inc. All rights reserved.

[illegible]

SAS® Intelligent Decisioning

Query Data Grids with SQL – Aggregate data

Output customers where the total amount per customer is over 150,000

revenue_all

Data Grid Formatted Plain

AMOUNT	CUST_ID
65000	A4711
152000	A5874
85000	A1234
25000	A1234
125000	A4711
74000	A4711

Close

revenue_pc

Data Grid Formatted Plain

CUST_ID	AMOUNT
A4711	264000
A5874	152000

SAS® Intelligent Decisioning

Query Data Grids with SQL – Aggregate data

Expression Editor: Default

Variables Functions

Filter

- revenue_all
- i
- dg
- revenue_pc
- cust_id
- buffer
- j
- amount

```
/*Output customers where the total
amount per customer is over 150,000 */
dataGrid_sort(revenue_all, 'cust_id', 'A', revenue_all)
i=1
buffer= ''
do while(i <= dataGrid_count(revenue_all))
  cust_id= dataGrid_getString(revenue_all, 'cust_id', i)
  if cust_id != buffer then do
    dataGrid_tableSelect(revenue_all, '*', 'cust_id = ' || cust_id , dg)
    amount= dataGrid_sum(dg, 'amount')
    if amount > 150000 then do
      j= dataGrid_addRow(revenue_pc)
      dataGrid_setString(revenue_pc, 'cust_id', j, cust_id)
      dataGrid_setDouble(revenue_pc, 'amount', j, amount)
    end
  end
  buffer= cust_id
  i= i+1
end
```

Verify syntax Clear

- 8 Variables
- 8 Data Grid functions
- 17 lines of code
- Comment to understand the code

Expression Editor: Default_assignment_1

Variables F >

Filter

- revenue_pc
- stmt
- msg
- revenue_all

AND OR NOT = < >

```
stmt= '
SELECT cust_id, sum(amount) as amount
FROM DG1 as revenue_all
GROUP BY cust_id
HAVING sum(amount) >= 150000
msg= dataGrid_sql(stmt, revenue_pc, revenue_all, null, null, null)
```

Verify syntax Clear

- 4 Variables
- 1 Data Grid (custom) function
- 1 SQL statement
- 2 lines of code

SAS® Intelligent Decisioning

Query Data Grids with SQL – Cartesian Product

Compare every row from table A with every row from table B

ID	Name	Code	Country
1	Alice	UK	United Kingdom
2	Bob	US	United States of America
		FR	France

Cross Join

ID	Name	Code	Country
1	Alice	UK	United Kingdom
1	Alice	US	United States of America
1	Alice	FR	France
2	Bob	UK	United Kingdom
2	Bob	US	United States of America
2	Bob	FR	France

SAS® Intelligent Decisioning

Query Data Grids with SQL – Cartesian Product

Customer situation: Check person on customs list against sanction list
Compare every row from list A with every row from list B

Challenge:

- 100 – 200 people on customs list
- ~10.000 people on sanction list
- 8 fields to check
- DS2 calling dataGrid functions
 - $100 * 10.000 * 8 * 2 = 8.000.000$
- Using DuckDB function
 - Submit one SQL statement

SAS® Intelligent Decisioning

Query Data Grids with SQL – Cartesian Product

Using Data Grid functions

```
1 package "${PACKAGE_NAME}" /inlines;
2 method execute(package datagrid blTbl,
3 package datagrid checkTbl,
4 in_out package datagrid resultTbl);
5
6 dcl integer first_name_mt last_name_mt email_mt street_mt city_mt phone_mt zip_mt range_mt;
7 dcl integer i j k check_cnt bl_cnt;
8
9 datagrid_clear(resultTbl);
10 datagrid_create(resultTbl, '{"metadata":[{"BL_ID":"Int"}, {"FIRST_NAME_MT":"Int"}, {"LAST_NAME_MT":"Int"}, {"EMAIL_MT":"Int"}, {"'
11
12 bl_cnt= datagrid_count(blTbl);
13 check_cnt= datagrid_count(checkTbl);
14
15 i= 1;
16 do while(i <= check_cnt);
17   j= 1;
18   do while(j <= bl_cnt);
19     first_name_mt= 0;
20     if datagrid_getString(checkTbl, 'first_name_mc', i) = datagrid_getString(blTbl, 'first_name_mc', j) then do;
21       first_name_mt= i;
22     end;
23     last_name_mt= 0;
24     if datagrid_getString(checkTbl, 'last_name_mc', i) = datagrid_getString(blTbl, 'last_name_mc', j) then do;
25       last_name_mt= i;
26     end;
27     email_mt= 0;
28     if datagrid_getString(checkTbl, 'email_mc', i) = datagrid_getString(blTbl, 'email_mc', j) then do;
29       email_mt= i;
30     end;
31     street_mt= 0;
32     if datagrid_getString(checkTbl, 'street_mc', i) = datagrid_getString(blTbl, 'street_mc', j) then do;
33       street_mt= i;
34     end;
35     city_mt= 0;
36     if datagrid_getString(checkTbl, 'city_mc', i) = datagrid_getString(blTbl, 'city_mc', j) then do;
37       city_mt= i;
38     end;
39     phone_mt= 0;
40     if datagrid_getString(checkTbl, 'phone_std', i) = datagrid_getString(blTbl, 'phone_std', j) then do;
41       phone_mt= i;
42     end;
43     zip_mt= 0;
44     if datagrid_getDouble(checkTbl, 'zip', i) = datagrid_getDouble(blTbl, 'zip', j) then do;
45       zip_mt= i;
46     end;
47     range_mt= 0;
48     if datagrid_getDouble(checkTbl, 'range', i) = datagrid_getDouble(blTbl, 'range', j) then do;
49       range_mt= i;
50     end;
51
52     if first_name_mt = 1 or last_name_mt = 1 or email_mt = 1 or street_mt = 1 or city_mt = 1 or phone_mt = 1 or zip_mt = 1
53       k= datagrid_addRow(resultTbl, i);
54       datagrid_setInt(resultTbl, 'BL_ID', k, datagrid_getDouble(blTbl, 'ID', j));
55       datagrid_setInt(resultTbl, 'first_name_mt', k, first_name_mt);
56       datagrid_setInt(resultTbl, 'last_name_mt', k, last_name_mt);
57       datagrid_setInt(resultTbl, 'email_mt', k, email_mt);
58       datagrid_setInt(resultTbl, 'street_mt', k, street_mt);
59       datagrid_setInt(resultTbl, 'city_mt', k, city_mt);
60       datagrid_setInt(resultTbl, 'phone_mt', k, phone_mt);
61       datagrid_setInt(resultTbl, 'zip_mt', k, zip_mt);
62       datagrid_setInt(resultTbl, 'range_mt', k, range_mt);
63     end;
64     j= j+1;
65   end;
66   i= i+1;
67 end;
68 end;
69 end;
```

Cop

Using DuckDB – CROSS JOIN

```
1 stmt= '
2 SELECT
3   bl.id as bl_id,
4   CASE
5     WHEN bl.first_name_mc = dg1.first_name_mc THEN 1
6     ELSE 0
7   END as first_name_mt,
8   CASE
9     WHEN bl.last_name_mc = dg1.last_name_mc THEN 1
10    ELSE 0
11  END as last_name_mt,
12  CASE
13    WHEN bl.email_mc = dg1.email_mc THEN 1
14    ELSE 0
15  END as email_mt,
16  CASE
17    WHEN bl.street_mc = dg1.street_mc THEN 1
18    ELSE 0
19  END as street_mt,
20  CASE
21    WHEN bl.city_mc = dg1.city_mc THEN 1
22    ELSE 0
23  END as city_mt,
24  CASE
25    WHEN bl.zip = dg1.zip THEN 1
26    ELSE 0
27  END as zip_mt,
28  CASE
29    WHEN bl.phone_std = dg1.phone_std THEN 1
30    ELSE 0
31  END as phone_mt,
32  CASE
33    WHEN bl.range = dg1.range THEN 1
34    ELSE 0
35  END as range_mt
36 FROM pg.blacklist as bl CROSS JOIN dg1 as dg1
37 WHERE (bl.first_name_mc = dg1.first_name_mc
38 OR bl.last_name_mc = dg1.last_name_mc)
39 OR bl.email_mc = dg1.email_mc
40 OR bl.street_mc = dg1.street_mc
41 OR bl.city_mc = dg1.city_mc
42 OR bl.phone_std = dg1.phone_std
43 OR bl.zip = dg1.zip
44 OR bl.range = dg1.range
45 ';
```



